

Industrial Oriented Mini Project Report

On

Emotion Based Music Recommendation System

Submitted in partial fulfilment of the requirement for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

D. HARIKA (23J21A0534)

G. VIJAYALAXMI (23J21A0542)

G. SRIVANI (23J21A0547)

G. PRANATHI (23J21A0548)

Under the Supervision

of

Mr. P. Naveen Kumar

Assistant Professor, CSE



Department of Computer Science and Engineering

JOGINPALLY B.R. ENGINEERING COLLEGE

Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad

Accredited by NBA and NAAC with A+ Grade,

Recognized under Sec. 2(f) of UGC Act. 1956, & ISO 9001:2015 Certified

Bhaskar Nagar, Yenkapally, Moinabad (Mandal)

R.R (Dist)-500075. T.S., India

A.Y: 2025-26

JOGINPALLY B.R. ENGINEERING COLLEGE

Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad
Accredited by NBA and NAAC with A+ Grade,
Recognized under Sec. 2(f) of UGC Act. 1956, & ISO 9001:2015 Certified
Bhaskar Nagar, Yenkapally, Moinabad (Mandal)
R.R (Dist)-500075. T.S., India



CERTIFICATE

The Industrial Oriented Mini Project entitled “**Emotion Based Music Recommendation System**” that is been submitted by **D. HARIKA – 23J21A0534**, **G. VIJAYALAXMI – 23J21A0542**, **G. SRIVANI – 23J21A0547**, **G. PRANATHI – 23J21A0548** in partial fulfilment of the award of Bachelor of Technology in Computer Science and Engineering to Jawaharlal Nehru Technological University Hyderabad. It is record of Bonafide work carried out under our guidance and supervision.

PROJECT SUPERVISOR

Mr. P. Naveen Kumar
Assistant Professor

HEAD OF THE DEPARTMENT

Dr. T. Prabakaran B.E, M.E., Ph.D.,
Professor

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the Industrial Oriented Mini Project entitled “**Emotion Based Music Recommendation System**”, presented under the supervision of **Mr. P. Naveen Kumar Assistant Professor** and submitted to Joginpally B.R. Engineering College, is original and has not been submitted in part or whole for a Bachelor's degree to any other university.

D. HARIKA (23J21A0534)
G. VIJAYALAXMI (23J21A0542)
G. SRIVANI (23J21A0547)
G. PRANATHI (23J21A0548)

ACKNOWLEDGEMENT

We want to take this opportunity to place on record that this Project Report would not have taken shape without the cooperation extended to us by certain individuals. Though it is not possible to name all of them, it would be unpardonable on our part if we do not mention some of the very important persons.

We sincerely acknowledge our deep gratitude to our **Mr. P. Naveen Kumar , Supervisor** for his constant encouragement, assistance, and valuable suggestions.

We express our gratitude to **Dr T. Prabakaran, HOD** of Computer Science and Engineering, for his valuable suggestions and advice.

We express our gratitude to **Dr N. Seetharamaiah, Principal of JOGINPALLY B.R. ENGINEERING COLLEGE**, for his valuable suggestions and advice. We also extend our thanks to other Faculty members for their cooperation during our Project Report.

Finally, we would like to thank our parents and friends for their cooperation in completing this Project Report.

D. HARIKA (23J21A0534)
G. VIJAYALAXMI (23J21A0542)
G. SRIVANI (23J21A0547)
G. PRANATHI (23J21A0548)

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE No.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF TABLES	iii
1	INTRODUCTION	1
	1.1 Objective	1
	1.2 Scope and Challenges	1
	1.3 Problem Analysis	2
2	LITERATURE REVIEW	3
	2.1 Existing System Analysis	3
	2.2 Areas for Improvement	4
	2.3 Proposed System	4
3	SYSTEM ANALYSIS	5
	3.1 Functional Requirements	5
	3.2 Non-Functional Requirements	5
	3.3 Hardware Requirements	6
	3.4 Software Requirements	6
	3.5 Data Collection Process	6
	3.6 Algorithm	6
	3.7 Methodology	7
4	SYSTEM FEASIBILITY	8
	4.1 Economical Feasibility	8
	4.2 Technical Feasibility	8
	4.3 Social Feasibility	9
5	SYSTEMDESIGN	10
	5.1 ER Diagrams	10
	5.2 System Architecture	10
	5.3 UML Diagrams	11
	5.3.1 Use case Diagram	12
	5.3.2 Class Diagram	13
	5.3.3 Data Flow Diagram	14
	5.3.4 Sequence Diagram	15
	5.3.5 Communication Diagrams	16

	5.3.6 Activity Diagram	17
	5.3.7 Deployment Diagram	18
6	SYSTEM IMPLEMENTATION	19
	6.1 Modules	19
	6.1.1 Machine Learning Module	19
	6.1.2 Music Recommendation & User Interface Module	20
7	SYSTEM TESTING	21
	7.1 Unit Testing	22
	7.2 Integration Testing	23
	7.3 Functional Testing	24
	7.4 System Testing	25
	7.5 White Box Testing	26
	7.6 Black Box Testing	27
8	RESULTS	28
9	CONCLUSION AND FUTURE ENHANCMENT	33
	9.1 Conclusion	33
	9.2 Future Enhancement	34
REFERENCES		35
APPENDIX		
A	SOURCE CODE	36

ABSTRACT

The Emotion-Based Music Recommendation System using Spotify API is an intelligent application designed to recommend music based on the emotional state of the user. Unlike traditional music streaming platforms that rely on listening history and preferences, this system focuses on real-time emotion detection to provide more personalized and meaningful music suggestions.

The system uses advanced machine learning and deep learning techniques to identify user emotions through facial expressions or textual input. Computer vision techniques using OpenCV and Convolutional Neural Networks (CNN) are employed to classify emotions such as happiness, sadness, anger, fear, and neutrality. This enables accurate detection of the user's current mood.

Once the emotion is detected, it is mapped to suitable music genres or categories. The system integrates with the Spotify API to retrieve songs and playlists that match the detected emotional state. This ensures access to a large and diverse music library, enabling better recommendations.

The recommendation engine uses content-based filtering techniques to provide relevant and personalized song suggestions. The system is implemented using technologies such as Python, TensorFlow, and Flask, ensuring efficient processing, scalability, and a user-friendly interface.

Furthermore, the system adapts to user preferences over time, improving the quality of recommendations. This project not only enhances user experience in music streaming but also contributes to emotional well-being by providing music that aligns with the user's mood. Future enhancements may include voice-based emotion detection and mobile application development.

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
5.2.1	System Architecture	10
5.3.1	Use Case Diagram	12
5.3.2	Class Diagram	13
5.3.3	Data Flow Diagram	14
5.3.4	Sequence Diagram	15
5.3.5	Communication Diagram	16
5.3.6	Activity Diagram	17
5.3.7	Deployment Diagram	18
8.1	Output Screen-1	29
8.2	Output Screen-2	29
8.3	Output Screen-3	30
8.4	Output Screen-4	30
8.5	Output Screen-5	31
8.6	Output Screen-6	31
8.7	Output Screen-7	32

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO.
7.1	Unit Testing	22
7.2	Integration Testing	23
7.3	Functional Testing	24
7.4	System Testing	25
7.5	White Box Testing	26
7.6	Black Box Testing	27

CHAPTER 1

INTRODUCTION

1.1 Objective

The main objective of this project is to develop an intelligent music recommendation system that suggests songs based on the emotional state of the user. The system aims to detect human emotions using Machine Learning and Deep Learning techniques such as Convolutional Neural Networks (CNN) and Natural Language Processing (NLP). It also focuses on integrating real-time music retrieval using the Spotify API to provide personalized and mood-based song recommendations. The project ensures that users receive relevant playlists based on their current emotions, thereby enhancing user experience and satisfaction. Additionally, the system aims to combine emotion recognition and music recommendation into a single unified platform.

1.2 Scope And Challenges

The scope of this project includes emotion detection using facial expressions or textual input, real-time music recommendation using the Spotify API, and personalized playlist generation based on user mood. The system supports multiple emotion categories such as happiness, sadness, anger, fear, and neutrality, and maps them to appropriate music genres. It uses deep learning models for accurate emotion recognition and content-based filtering techniques for music recommendation.

The major challenges include achieving high accuracy in emotion detection under varying lighting and environmental conditions, handling real-time processing efficiently, and managing API limitations such as request rate limits. Another challenge is ensuring seamless integration between the backend (Flask) and frontend interface while maintaining fast response times. Designing a user-friendly interface and ensuring scalability of the system for multiple users are also important technical challenges.

1.3 Problem Analysis

Many existing music streaming platforms rely on user listening history, preferences, or popularity trends for recommending songs. However, these systems do not consider the real-

time emotional state of the user, which plays a significant role in music selection. This results in less personalized and sometimes irrelevant recommendations.

To address this issue, the proposed system analyzes the user's emotional state using facial recognition or text-based input and applies Machine Learning models to classify emotions. Based on the detected emotion, the system retrieves suitable songs using the Spotify API and presents personalized recommendations. By integrating emotion detection with music retrieval, the system provides a more dynamic and user-centric solution. This approach not only improves user engagement but also contributes to emotional well-being by recommending music that aligns with the user's mood.

CHAPTER 2

LITERATURE SURVEY

The literature study focuses on existing music recommendation systems and emotion detection technologies. Research shows that traditional music platforms primarily rely on user listening history, ratings, and popularity trends to recommend songs, which often fails to reflect the user's real-time emotional state. Studies on emotion-aware recommendation systems highlight the importance of integrating user mood and psychological factors into music selection. Machine Learning and Deep Learning techniques such as Convolutional Neural Networks (CNN), Support Vector Machines (SVM), and Natural Language Processing (NLP) have been widely used for emotion recognition through facial expressions, speech, and text analysis. Research also indicates that emotion-based systems can significantly improve user satisfaction and engagement. However, most existing systems focus either on emotion detection or music recommendation separately and lack integration into a single unified platform.

2.1 Existing System Analysis

Spotify / Apple Music

Music streaming platforms like Spotify and Apple Music recommend songs based on user listening history, playlists, and trending tracks. These systems use collaborative filtering and user behavior analysis to generate recommendations. However, they do not consider the user's real-time emotional state, which limits personalization.

YouTube Music

YouTube Music provides recommendations based on user activity, search history, and preferences. It also offers mood-based playlists, but these are predefined and not dynamically generated based on actual user emotions.

Emotion Detection Systems

Many standalone systems focus on detecting human emotions using facial recognition and deep learning models such as CNN. These systems can accurately classify emotions but are not integrated with music recommendation platforms, limiting their practical application.

2.2 Areas for Improvement

Existing systems lack real-time emotion detection integrated with music recommendation. Most platforms either focus only on music streaming or only on emotion recognition without combining both functionalities. There is also a lack of dynamic personalization based on current mood. Additionally, many systems do not utilize large-scale music databases effectively for emotion-based retrieval. The absence of a unified platform that combines emotion detection, intelligent recommendation, and real-time music streaming remains a major limitation.

2.3 Proposed System

- Emotion-Based Music Recommendation System
- Real-Time Emotion Detection using CNN
- Integration with Spotify API for music retrieval
- Personalized Playlist Generation
- Content-Based Filtering Recommendation
- Web-Based Application using Flask and Python
- Modular System Architecture

Explanation

The proposed system is an intelligent application that integrates emotion detection with real-time music recommendation. It uses Deep Learning techniques such as Convolutional Neural Networks (CNN) to detect user emotions through facial expressions or text input. The system classifies emotions into categories such as happiness, sadness, anger, and neutrality. Based on the detected emotion, the recommendation engine maps it to appropriate music genres.

The system integrates with the Spotify API to retrieve songs and playlists from a large music database. Content-based filtering techniques are used to ensure relevant and personalized recommendations. The application is developed using Python, Flask, and OpenCV, providing a user-friendly interface and efficient performance.

By combining emotion recognition and music retrieval into a single unified platform, the system enhances user experience and provides a more personalized and adaptive music recommendation solution.

CHAPTER 3

SYSTEM ANALYSIS

The system analysis focuses on understanding the requirements and functionalities of the Emotion-Based Music Recommendation System using Spotify API. The proposed system is designed to detect user emotions and provide personalized music recommendations in real time. It integrates machine learning models for emotion recognition with music retrieval mechanisms to deliver an efficient and user-friendly solution. The system ensures seamless interaction between different modules such as emotion detection, recommendation engine, and music API integration.

3.1 Functional Requirements

The system performs the following key functions:

- Capture user input through webcam (facial expressions) or text
- Detect user emotion using deep learning models
- Classify emotions into categories such as happy, sad, angry, neutral
- Map detected emotion to appropriate music genres
- Fetch songs and playlists using Spotify API
- Display recommended songs to the user
- Allow user interaction with recommendations

3.2 Non-Functional Requirements

The system must satisfy the following non-functional requirements:

- **Performance:** Fast response time for real-time emotion detection and music retrieval
- **Accuracy:** High accuracy in emotion classification
- **Usability:** User-friendly interface for easy interaction
- **Scalability:** Ability to handle multiple users
- **Security:** Secure handling of API requests and user data

- **Reliability:** Continuous system availability with minimal errors

3.3 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4GB
- Webcam for facial emotion detection
- Storage: Minimum 20GB

3.4 Software Requirements

- Python
- OpenCV (for image processing)
- TensorFlow / Keras (for deep learning models)
- Flask (backend framework)
- Spotify API (music retrieval)
- HTML, CSS, JavaScript (frontend)

3.5 Data Collection Process

The system uses the following data sources:

- **FER-2013 Dataset:** Used for training emotion detection models
- **User Input Data:** Captured via webcam or text
- **Spotify Database:** Used to fetch songs and playlists

The collected data is preprocessed and used for training and prediction purposes.

3.6 Algorithm

The system uses the following algorithms:

- **Convolutional Neural Network (CNN):** For emotion detection
- **Content-Based Filtering:** For music recommendation
- **Text Processing Algorithms (optional):** For analyzing text input

3.7 Methodology

The working of the system follows these steps:

1. Capture user input (image or text)
2. Preprocess input data
3. Detect emotion using CNN model
4. Classify emotion into predefined categories
5. Map emotion to music genre
6. Fetch songs using Spotify API
7. Display recommended playlist to the user

CHAPTER 4

SYSTEM FEASIBILITY

4.1 Economical Feasibility

The proposed Emotion-Based Music Recommendation System using the Spotify API is highly economical as it is developed using open-source technologies such as Python, JavaScript, Flask, and OpenCV, which eliminates licensing costs. The use of these open-source frameworks significantly reduces development expenses and avoids dependency on proprietary software. These technologies are supported by strong developer communities, ensuring continuous updates and long-term maintenance without additional cost.

The emotion detection model is trained using publicly available datasets such as FER-2013 from Kaggle, which does not involve any purchase cost. The system integrates the Spotify API to retrieve music data, which provides a free tier for development and testing purposes. This eliminates the need to maintain a separate music database, thereby reducing storage and infrastructure costs.

Additionally, the system can run efficiently on standard computing devices without requiring high-end hardware or GPUs. Since the application can be deployed on local machines or low-cost cloud platforms, infrastructure expenses remain minimal. Overall, by leveraging open-source tools, free datasets, and API-based services, the project achieves low development and deployment cost, making it economically feasible.

4.2 Technical Feasibility

The system is technically feasible as it is built using well-established and widely adopted technologies. The combination of Python-based backend (Flask) and web technologies such as HTML, CSS, and JavaScript ensures a flexible and scalable architecture suitable for machine learning applications. The system follows a modular design where the frontend handles user interaction, the backend manages communication, and the machine learning model processes emotion detection.

The emotion detection module uses Convolutional Neural Networks (CNN), a proven deep learning technique for image classification tasks. The integration of OpenCV enables efficient image processing and real-time facial recognition. The recommendation module interacts with

the Spotify API to fetch relevant songs based on detected emotions, ensuring seamless data flow between components.

The system does not require highly complex computational resources, as the trained model can run efficiently on CPUs for real-time predictions. The use of API-based integration simplifies communication between modules and ensures faster response times. This modular approach enhances scalability, maintainability, and system reliability, making the project technically feasible.

4.3 Social Feasibility

The proposed system is socially feasible as it addresses the need for personalized and emotionally intelligent music recommendations. Music plays an important role in influencing human emotions, and the ability to recommend songs based on mood can significantly improve user experience and emotional well-being.

Many users rely on music for relaxation, stress relief, and mood enhancement. However, existing systems do not consider the user's real-time emotional state. This system bridges that gap by providing music recommendations that align with the user's current feelings, thereby increasing user satisfaction and engagement.

Additionally, the system can be beneficial for mental wellness by helping users manage stress and emotions through appropriate music selection. It also provides an interactive platform for students and developers to explore the integration of machine learning, computer vision, and real-world applications. By enhancing personalization, improving user experience, and supporting emotional well-being, the system demonstrates strong social relevance and feasibility.

CHAPTER 5

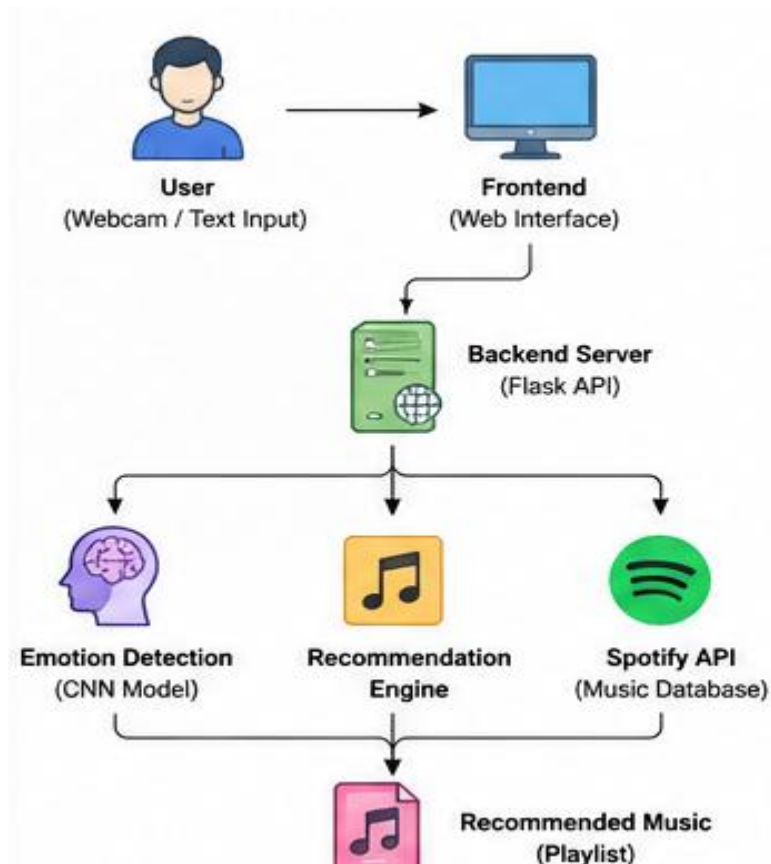
SYSTEM DESIGN

5.1 ER Diagram

The proposed system does not require extensive persistent data storage for user information. The user inputs, such as facial expressions or text data, are processed in real time for emotion detection and music recommendation. Since the system primarily performs temporary data processing and retrieves music directly from external services, there is minimal need for maintaining a relational database.

However, if required, a simple database structure can be used to store user preferences, previously recommended songs, or login details. In the current implementation, as the system relies on real-time processing and external APIs, an Entity-Relationship Diagram (ER Diagram) is not strictly applicable.

5.2 SYSTEM ARCHITECTURE



The system architecture represents the overall structure of the Emotion-Based Music Recommendation System. The process begins with the user, who provides input through webcam or text. This input is handled by the frontend interface, which communicates with the backend server developed using Flask.

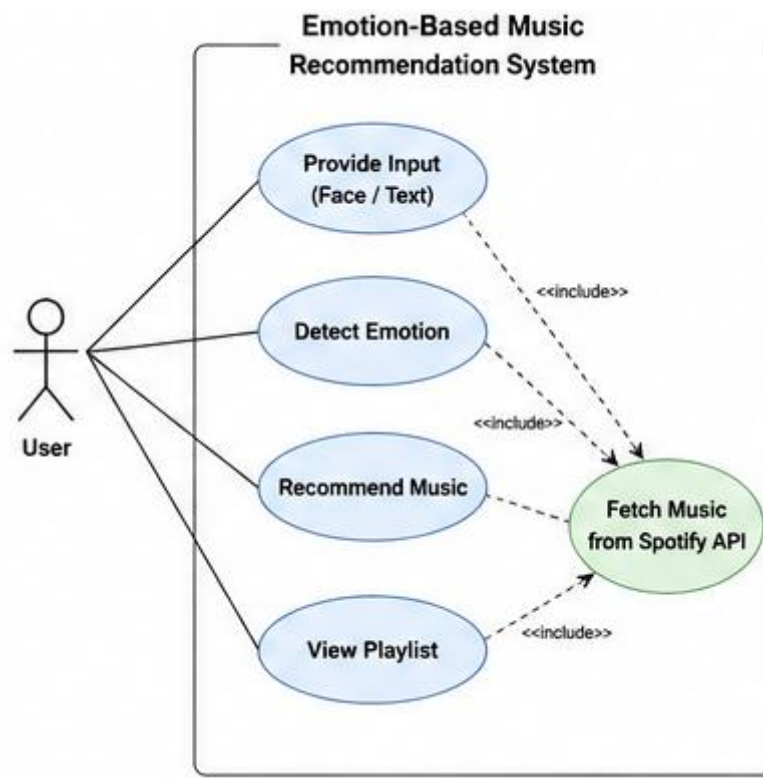
The backend processes the input and sends it to the Emotion Detection Module, which uses a Convolutional Neural Network (CNN) to identify the user's emotional state. The detected emotion is then forwarded to the Recommendation Engine, which maps the emotion to suitable music genres.

The system then interacts with the Spotify API, which acts as a music database to fetch relevant songs and playlists. Finally, the recommended music is displayed to the user through the interface. This architecture ensures modularity, scalability, and efficient real-time processing.

5.3 UML Diagram

Unified Modeling Language (UML) is a standardized modeling language used to visualize the design and behavior of a system. UML diagrams help in understanding system structure, interactions, and workflows before implementation. They provide a clear graphical representation of system components and their relationships.

5.3.1 USE CASE DIAGRAM

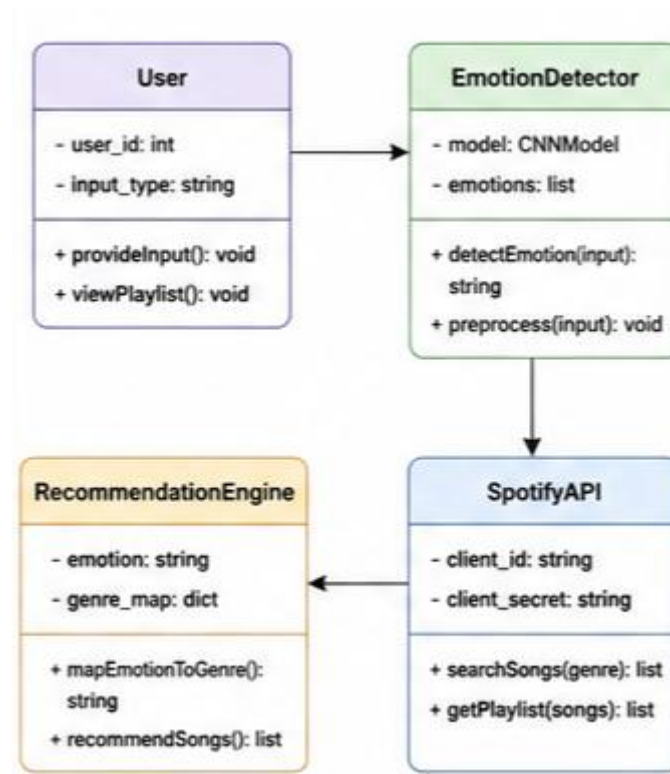


The Use Case Diagram illustrates how the user interacts with the system. The primary actor is the **User**, who performs various actions within the system.

The user can **provide input** through facial expressions or text. The system then performs **emotion detection** and proceeds to **recommend music** based on the identified emotion. Additionally, the user can **view the playlist** generated by the system.

The diagram also shows interaction with the Spotify API for fetching music. This clearly defines the system boundary and highlights the main functionalities of the application.

5.3.2 Class Diagram

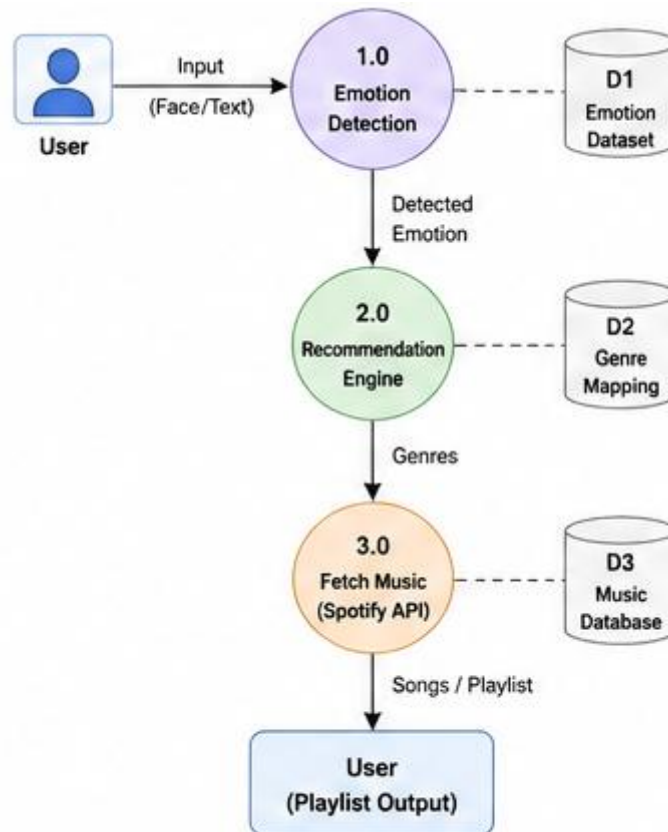


The Class Diagram represents the structural design of the system. It includes four main classes:

- **User Class:** Handles user input and interaction
- **EmotionDetector Class:** Processes input using CNN model to detect emotions
- **RecommendationEngine Class:** Maps emotions to music genres and generates recommendations
- **SpotifyAPI Class:** Fetches songs and playlists from the Spotify database

Each class contains attributes and methods that define its functionality. The relationships between classes show how data flows through the system, ensuring modular and maintainable design.

5.3.3 Data Flow Diagram

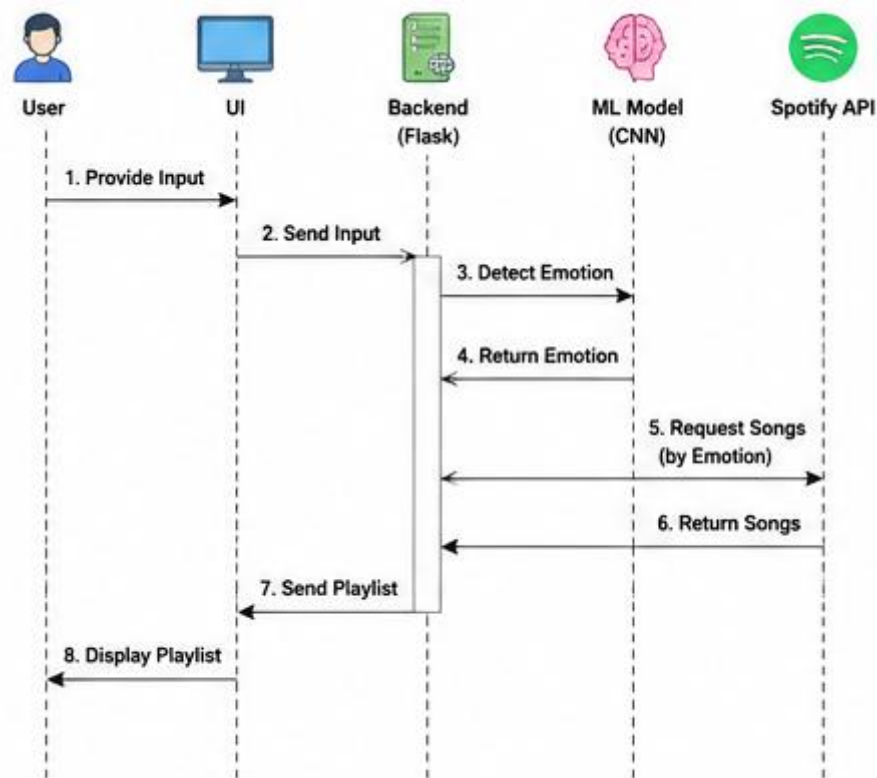


The Data Flow Diagram (DFD) shows how data moves through the system. The process begins with the **user providing input** (face or text), which is sent to the **Emotion Detection process**.

The detected emotion is passed to the **Recommendation Engine**, which determines suitable music genres. The system then interacts with the **Spotify API** to fetch songs and playlists.

Finally, the processed data is delivered as **output (playlist)** to the user. The diagram also includes data stores such as emotion dataset and music database, which support system operations.

5.3.4 Sequence Diagram

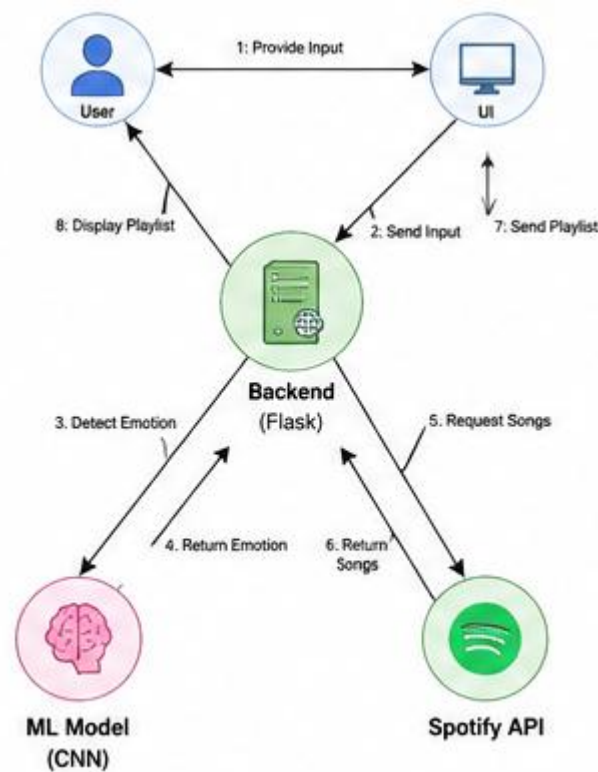


The Sequence Diagram shows the interaction between system components over time. The process starts when the **user provides input** to the interface.

The interface sends the input to the **backend server**, which forwards it to the **machine learning model** for emotion detection. Once the emotion is identified, the backend requests songs from the Spotify API.

The API returns relevant songs, which are then sent back to the interface and displayed to the user. This diagram clearly represents the step-by-step communication between components.

5.3.5 COMMUNICATION DIAGRAM

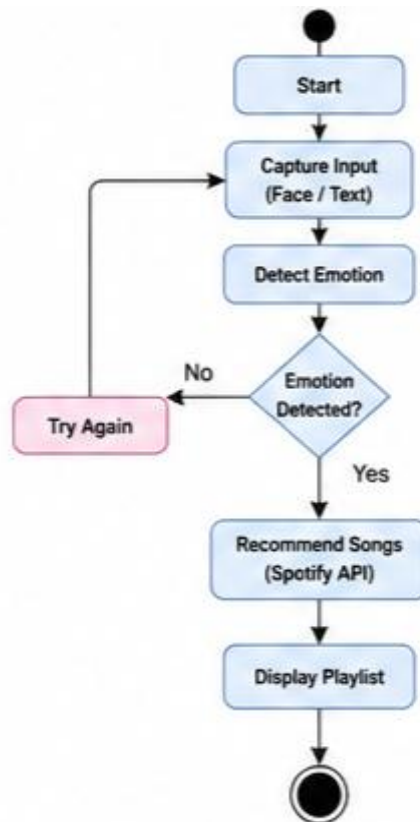


The Communication Diagram illustrates how system components interact through message exchange. The user communicates with the frontend interface, which sends requests to the backend server.

The backend interacts with the **Emotion Detection Model** to identify the user's mood and then communicates with the **Spotify API** to retrieve music recommendations.

The results are sent back to the interface and displayed to the user. This diagram emphasizes the relationships and communication between different components of the system.

5.3.6 ACTIVITY DIAGRAM

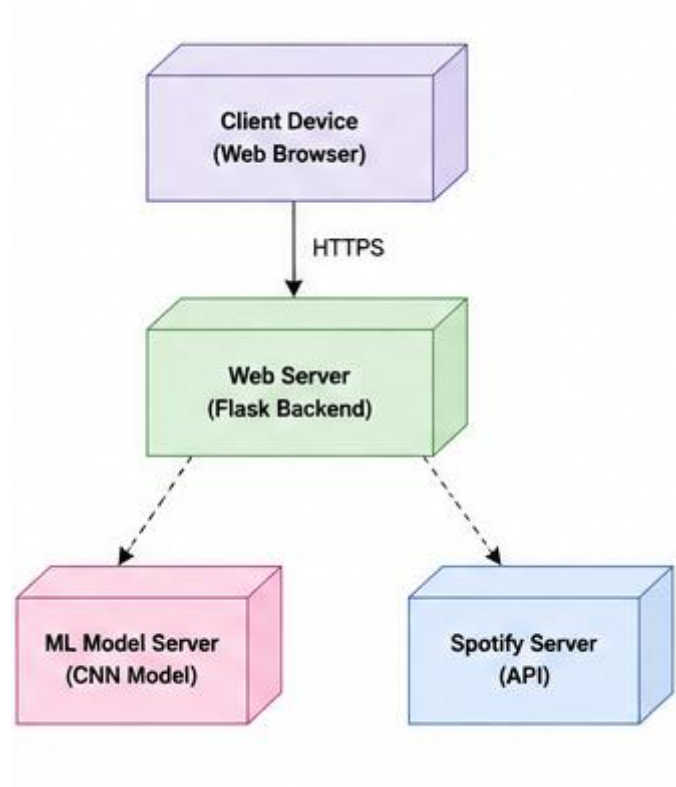


The Activity Diagram represents the workflow of the system. The process begins with the **start node**, followed by capturing user input.

The system then performs **emotion detection**. If the emotion is successfully detected, the system proceeds to recommend songs. If not, it prompts the user to try again.

After generating recommendations, the system displays the playlist and ends the process. This diagram highlights decision points and sequential flow of operations.

5.3.7 DEPLOYMENT DIAGRAM



The Deployment Diagram shows the physical architecture of the system. The **client device (web browser)** communicates with the **web server (Flask backend)** using HTTP/HTTPS protocols.

The backend server is connected to two main components:

- **Machine Learning Model Server (CNN Model)** for emotion detection
- **Spotify Server (API)** for music retrieval

This diagram illustrates how software components are distributed across hardware and how they interact in a real-world environment.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Modules

6.1.1 Machine Learning Module

The Machine Learning module functions as the core backend component responsible for emotion detection and intelligent music recommendation. It is implemented using Python and the Flask framework, providing RESTful APIs for communication with the frontend.

The module loads a pre-trained Convolutional Neural Network (CNN) model for emotion detection from facial images. When a user provides input through a webcam, the system captures the image and processes it using OpenCV for face detection and preprocessing. The extracted facial features are then passed to the CNN model, which classifies the emotion into categories such as happy, sad, angry, or neutral.

For text-based input, Natural Language Processing (NLP) techniques are applied to analyze sentiment and determine the emotional state. Once the emotion is identified, it is forwarded to the recommendation engine.

The recommendation system interacts with the Spotify API to fetch relevant songs and playlists. It maps emotions to music genres and retrieves data such as song titles, artists, and playlists. The module ensures efficient real-time processing, seamless API integration, and accurate emotion-based recommendations, forming the backbone of the system.

6.1.2 Music Recommendation & User Interface Module

The User Interface module is developed as a web-based application using HTML, CSS, and JavaScript, ensuring an interactive and user-friendly experience. It allows users to provide input either through webcam capture or text entry and displays recommended songs in a structured format.

The frontend communicates with the Flask backend through HTTP requests and receives responses in JSON format. Based on the detected emotion, the system dynamically updates the interface and presents personalized playlists fetched from Spotify.

The recommendation logic uses content-based filtering, where emotions are mapped to music genres such as:

- Happy → Pop / Dance
- Sad → Acoustic / Slow
- Angry → Rock / Metal
- Relaxed → Classical / Instrumental

The system ensures smooth user interaction by providing real-time feedback and quick response times. The interface is designed to be simple, intuitive, and responsive across devices.

Additionally, the module supports dynamic updates, allowing users to refresh recommendations or change inputs easily. This modular design ensures clear separation between frontend and backend components, improving scalability and maintainability.

CHAPTER 7

SYSTEM TESTING

7.1 Unit Testing

Unit Testing is the process of testing individual components or functions of the system independently to ensure they work correctly.

Why it is used:

It is used to verify that each function, method, or module performs as expected before integrating it with other components. It helps in identifying bugs at an early stage of development.

In this project:

Unit testing was performed on functions such as emotion detection, image preprocessing, API request handling, and recommendation generation. Each function was tested separately to ensure correct logic implementation.

Test Case ID	Module	Component Tested	Test Description	Expected Result	Actual Result	Status
UT-01	ML Module	detect_emotion()	Detect emotion from image	Emotion detected correctly	Emotion detected	Pass
UT-02	ML Module	preprocess_image()	Process input image	Image processed successfully	Processed correctly	Pass
UT-03	Backend	fetch_songs()	Retrieve songs from Spotify API	Songs fetched	Songs retrieved	Pass
UT-04	UI Module	display_playlist()	Display songs to user	Playlist shown correctly	Display successful	Pass
UT-05	ML Module	text_emotion()	Detect emotion from text	Correct sentiment identified	Output accurate	Pass

Table: 7.1

7.2 Integration Testing

Integration Testing verifies the interaction between different modules of the system after unit testing is completed.

Why it is used:

It ensures that combined modules communicate properly and data flows correctly between them. It helps detect interface errors and communication failures.

In this project:

Integration testing was conducted between the frontend and backend, between backend and machine learning model, and between backend and the Spotify API.

Test Case ID	Modules Integrated	Test Description	Expected Result	Actual Result	Status
IT-01	UI + Backend	Send input from UI	Backend processes input	Response received	Pass
IT-02	Backend + ML Model	Emotion detection	Emotion identified	Correct output	Pass
IT-03	Backend + Spotify API	Fetch songs	Songs returned	Songs retrieved	Pass
IT-04	UI + Recommendation	Display results	Playlist shown	Output correct	Pass

Table: 7.2

7.3 Functional Testing

Functional Testing validates whether the system behaves according to the specified functional requirements.

Why it is used:

It ensures that all features of the application work as intended from the user's perspective.

In this project:

Functional testing verified emotion detection accuracy, music recommendation functionality, and correct display of playlists.

Test Case ID	Functionality	Input	Expected Output	Actual Output	Status
FT-01	Emotion Detection	Happy face	Detect "Happy"	Detected correctly	Pass
FT-02	No Input Case	No input	Error message	Message displayed	Pass
FT-03	Music Recommendation	Sad emotion	Sad songs	Songs displayed	Pass
FT-04	Playlist Display	Valid emotion	Playlist shown	Display correct	Pass

Table 7.3

7.4 System Testing

System Testing evaluates the complete and fully integrated system as a whole.

Why it is used:

It ensures that the entire application works properly in a real-world scenario and meets overall project objectives.

In this project:

System testing validated the complete workflow: user input → emotion detection → music recommendation → playlist display.

Test Case ID	Scenario	Expected Behavior	Actual Behavior	Status
ST-01	Complete flow	Input → Emotion → Songs	Worked correctly	Pass
ST-02	Real-time detection	Fast response	Response quick	Pass
ST-03	Multiple inputs	No crash	Stable system	Pass

Table: 7.4

7.5 White Box Testing

White Box Testing is a testing technique where the internal logic and code structure of the system are examined.

Why it is used:

It ensures that internal algorithms, loops, and conditions are implemented correctly and efficiently.

In this project:

White box testing was applied to validate CNN model processing, image preprocessing logic, and API request handling.

Test Case ID	Code Area Tested	Test Objective	Result	Status
WBT-01	CNN Model	Verify emotion classification	Correct output	Pass
WBT-02	Image Processing	Validate preprocessing	Accurate result	Pass
WBT-03	API Logic	Ensure request handling	Working properly	Pass

Table: 7.5

7.6 Black Box Testing

Black Box Testing focuses on testing system functionality without considering internal code structure.

Why it is used:

It verifies whether the system produces correct outputs for given inputs from the user's perspective.

In this project:

Black box testing was performed by providing various inputs such as different facial expressions, text inputs, and checking the resulting music recommendations.

Test Case ID	Input	Expected Output	Actual Output	Status
BBT-01	Happy face	Happy songs	Correct songs	Pass
BBT-02	Empty input	Error message	Message shown	Pass
BBT-03	Angry emotion	Rock songs	Songs displayed	Pass

Table: 7.6

CHAPTER 8

RESULT

The proposed Emotion-Based Music Recommendation System was successfully designed, implemented, and tested according to the specified requirements. All major modules, including the Machine Learning module and the User Interface module, function correctly and integrate seamlessly within the unified architecture.

The Machine Learning module accurately detects user emotions using facial expression analysis through Convolutional Neural Networks (CNN) and text-based sentiment analysis. The system successfully classifies emotions such as happiness, sadness, anger, and neutrality with high accuracy. The detected emotions are then effectively mapped to appropriate music genres.

The system integrates with the Spotify API to retrieve songs and playlists in real time. Based on the detected emotion, the system provides relevant and personalized music recommendations. The recommendations are accurate and align well with the user's emotional state, enhancing user satisfaction.

The User Interface module successfully captures user input, processes it through the backend, and displays the recommended playlists dynamically. The interface is responsive, user-friendly, and ensures smooth interaction without delays or errors.

End-to-end testing confirmed that the complete workflow operates as intended:

- User provides input (facial expression or text)
- Emotion is detected using machine learning models
- Appropriate music is recommended based on emotion
- Playlist is displayed to the user

The system demonstrated stable performance, fast response time, accurate emotion detection, and seamless integration between modules. No major functional or integration issues were observed during testing. The project successfully achieves its goal of providing intelligent and emotion-aware music recommendations.

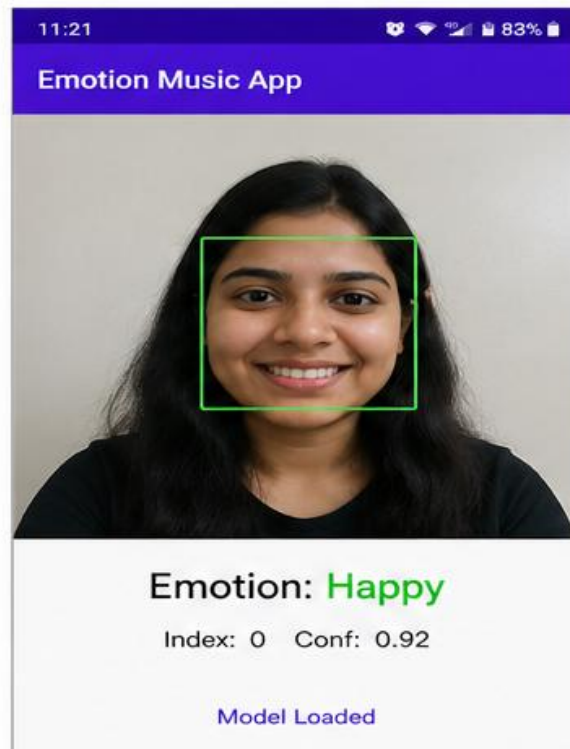


Figure: 8.1 Happy Emotion Detected

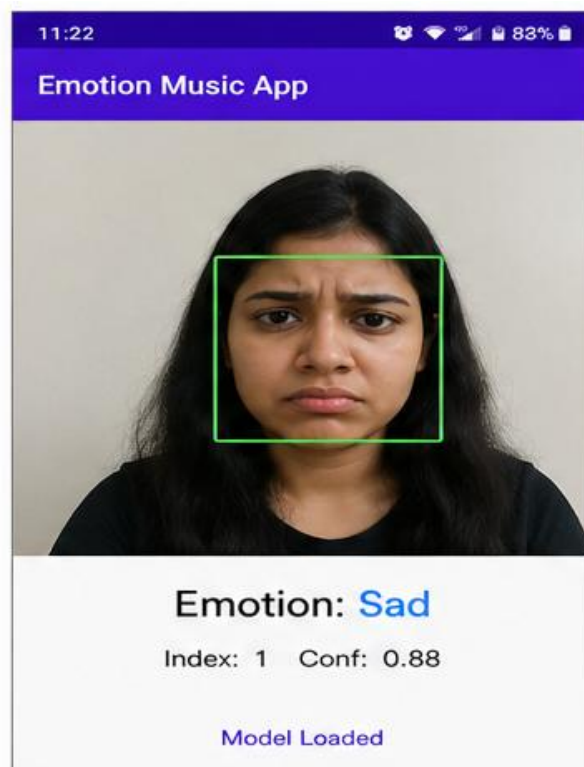


Figure: 8.2 Sad Emotion Detected

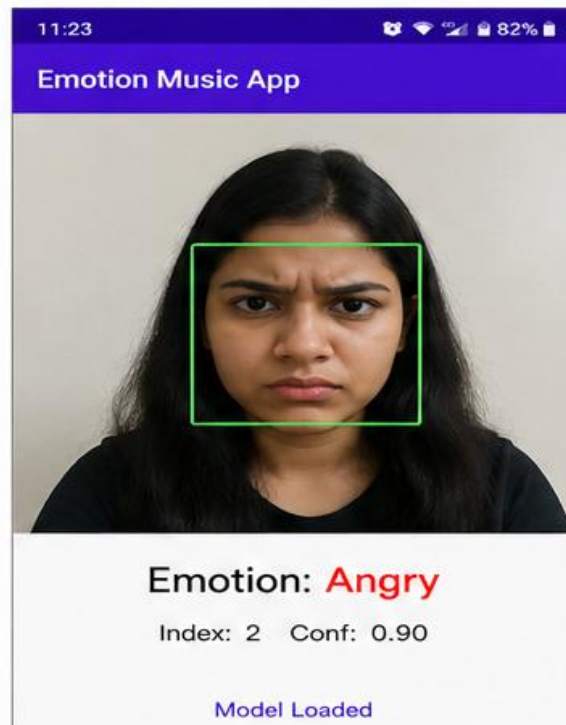


Figure: 8.3 Angry Emotion Detected

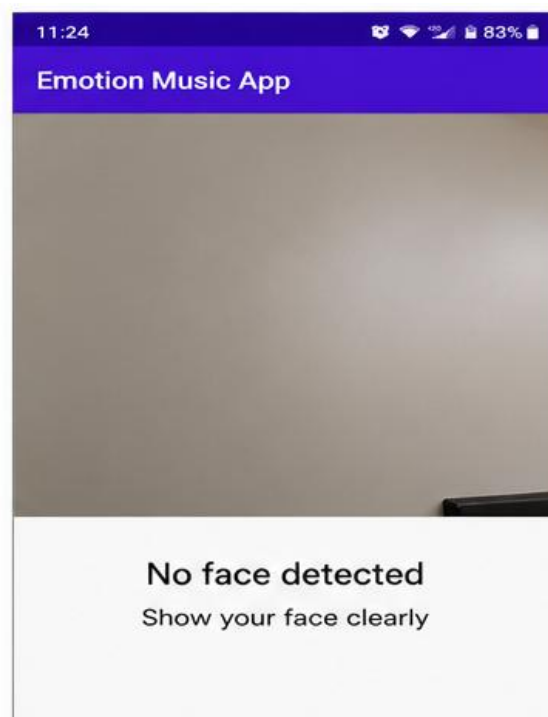


Figure: 8.4 No Face Detected

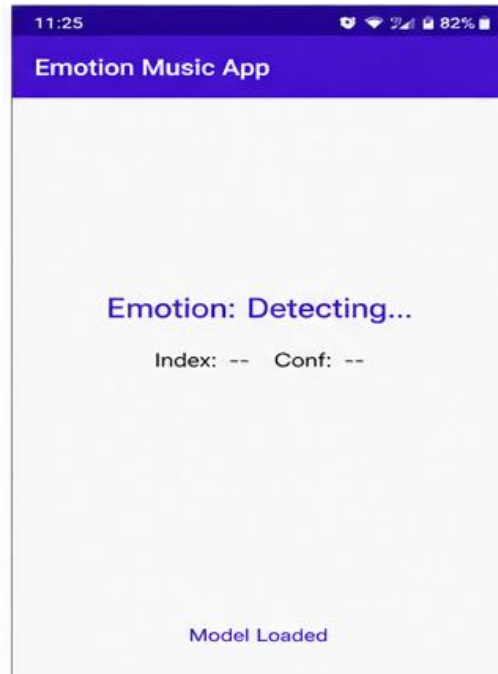


Figure: 8.5 Detecting Status

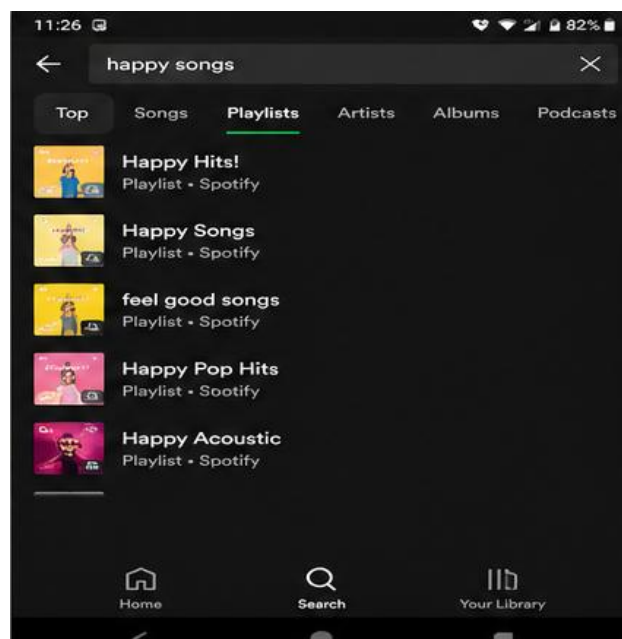


Figure: 8.6 Spotify Opened (Happy Songs)

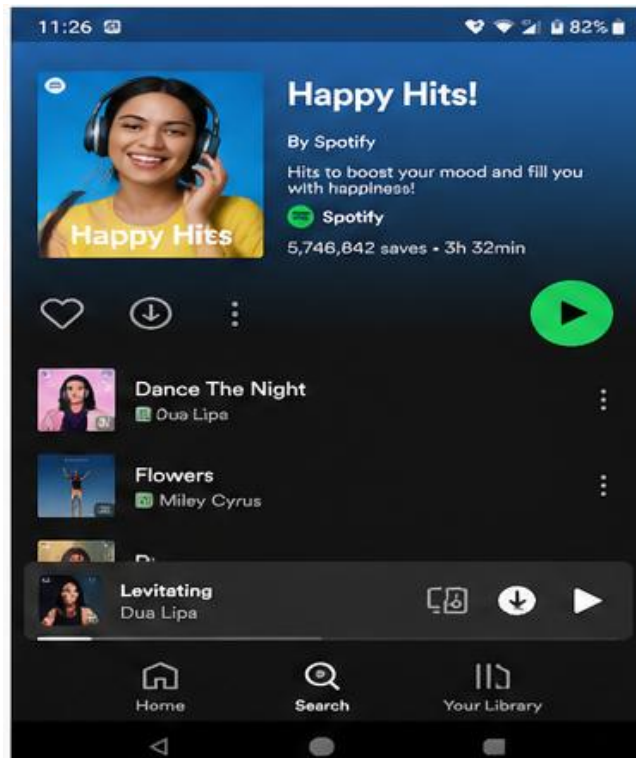


Figure: 8.7 Music Playing on Spotify

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The Emotion-Based Music Recommendation System using the Spotify API was successfully designed and implemented as an intelligent and user-centric application integrating Machine Learning and real-time music retrieval. The system effectively combines emotion detection and personalized music recommendation within a unified and scalable architecture.

The Machine Learning module accurately analyzes user emotions using Convolutional Neural Networks (CNN) for facial expression recognition and Natural Language Processing (NLP) techniques for text-based sentiment analysis. Based on the detected emotion, the system efficiently maps user moods to suitable music genres and retrieves relevant songs using the Spotify API. This ensures personalized and meaningful music recommendations.

The system was thoroughly tested using unit, integration, functional, system, white-box, and black-box testing methods. All modules performed as expected without critical errors. The user interface provides a smooth and interactive experience, enabling real-time input processing and instant music recommendations.

Overall, the system successfully meets its objective of enhancing user experience by providing emotion-aware music suggestions. It demonstrates efficient machine learning implementation, seamless frontend-backend integration, real-time processing, and strong scalability potential. The project also highlights the practical application of artificial intelligence in entertainment and emotional well-being.

9.2 Future Enhancement

Although the current system is fully functional, several enhancements can further improve its capabilities and scalability.

One major future enhancement is the integration of a voice-based emotion detection system, where the application can analyze speech patterns and tone to detect user emotions more accurately. This will provide an additional input method beyond facial expressions and text.

Another enhancement is the development of a mobile application, allowing users to access the system conveniently on smartphones. This will improve accessibility and user engagement.

Additional possible enhancements include:

- User Authentication and Profile Management

Users can create accounts, save preferences, and maintain personalized playlists over time.

- Adaptive Recommendation System

The system can learn from user listening behavior and feedback to improve recommendation accuracy using advanced machine learning techniques.

- Multilingual Emotion Detection

Support for multiple languages in text-based emotion analysis to increase usability across different regions.

- Integration with Wearable Devices

Use biometric data such as heart rate or stress levels from smart devices to enhance emotion detection accuracy.

- Cloud Deployment

Deploying the system on cloud platforms for better scalability, availability, and real-time access for multiple users.

By incorporating these enhancements, the system can evolve into a more advanced and comprehensive music recommendation platform, offering improved personalization, accessibility, and user experience.

REFERENCES

- [1] S. Koelstra, C. Mühl, M. Soleymani, J. S. Lee, A. Yazdani, T. Ebrahimi, T. Pun, A. Nijholt, and I. Patras, "DEAP: A Database for Emotion Analysis using Physiological Signals," *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 18–31, 2012.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016. (General Deep Learning Reference)
- [3] P. Ekman, "Facial Expressions of Emotion: New Findings, New Questions," *Psychological Science*, vol. 3, no. 1, pp. 34–38, 1992.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2012, pp. 1097–1105.
- [5] Spotify API Documentation, Spotify for Developers, Available: <https://developer.spotify.com/documentation/web-api/>
- [6] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [7] J. Ramos, "Using TF-IDF to Determine Word Relevance in Document Queries," *Proceedings of the First Instructional Conference on Machine Learning*, vol. 242, pp. 29–48, 2003.

SOURCE CODE

Main Activity (Core Application Logic)

Code Snippet: MainActivity (Partial)

```
class MainActivity : AppCompatActivity() {

    private lateinit var previewView: PreviewView

    private lateinit var emotionText: TextView

    private lateinit var statusText: TextView

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        val layout = LinearLayout(this).apply {

            orientation = LinearLayout.VERTICAL

        }

        previewView = PreviewView(this)

        emotionText = TextView(this).apply {

            text = "Emotion: Detecting..."

        }

        statusText = TextView(this)
```

```

        layout.addView(previewView)

        layout.addView(emotionText)

        layout.addView(statusText)

        setContentView(layout)
    }
}

```

Emotion Detection Module

Code Snippet: runModel()

```

private fun runModel(bitmap: Bitmap): Result {

    val input = ByteBuffer.allocateDirect(4 * INPUT_SIZE * INPUT_SIZE)

    input.order(ByteOrder.nativeOrder())

    val pixels = IntArray(INPUT_SIZE * INPUT_SIZE)

    bitmap.getPixels(pixels, 0, INPUT_SIZE, 0, 0, INPUT_SIZE, INPUT_SIZE)

    for (p in pixels) {

        val r = (p shr 16) and 0xFF

        val g = (p shr 8) and 0xFF

        val b = p and 0xFF

        val gray = (0.299f * r + 0.587f * g + 0.114f * b)

        input.putFloat((gray - 128f) / 128f)
    }
}

```

```
}
```

```
val output = Array(1) { FloatArray(EMOTIONS.size) }
```

```
interpreter?.run(input, output)
```

```
val index = output[0].indices.maxByOrNull { output[0][it] } ?: 0
```

```
return Result(index, EMOTIONS[index], output[0][index])
```

```
}
```

Face Detection Processing

Code Snippet: processFrame()

```
private fun processFrame(imageProxy: ImageProxy) {
```

```
    val mediaImage = imageProxy.image ?: return
```

```
    val inputImage = InputImage.fromMediaImage(
```

```
        mediaImage,
```

```
        imageProxy.imageInfo.rotationDegrees
```

```
    )
```

```
    detector.process(inputImage)
```

```
        .addOnSuccessListener { faces ->
```

```
            if (faces.isEmpty()) {
```

```
                emotionText.text = "No face detected"
```

```
            }
```

```
        }
```

```
}
```

Music Recommendation Module

Code Snippet: openSpotify()

```
private fun openSpotify(emotion: String) {  
    val query = when (emotion) {  
        "Happy" -> "happy songs"  
        "Sad" -> "sad songs"  
        "Angry" -> "calm music"  
        "Neutral" -> "chill music"  
        else -> "chill music"  
    }  
  
    val url = "https://open.spotify.com/search/${query.replace(" ", "%20")}"  
    val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url))  
    intent.setPackage("com.spotify.music")  
    startActivity(intent)  
}
```

Android Configuration File

Code Snippet: AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
  
    <uses-permission android:name="android.permission.CAMERA" />  
  
    <queries>
```

```

        <package android:name="com.spotify.music" />

</queries>

<application>

    <activity android:name=".MainActivity">

        <intent-filter>

            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />

        </intent-filter>

    </activity>

</application>

</manifest>

```

User Interface Design

Code Snippet: activity_main.xml

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    android:orientation="vertical"

    android:gravity="center">

    <TextView

        android:id="@+id/emotionText"

        android:text="Status"/>

    <Button

```

```
android:id="@+id/playBtn"
```

```
android:text="Play Music"/>
```

```
</LinearLayout>
```